Theorem 12

Example

- L_1 and L_2 are regular languages defined by the regular expressions below.

  $r_1 = (a + b)^*aa(a + b)^*$

  $r_2 = b^*(ab^*ab^*)^*$

- A word in L_2 can have some b's in the front
- But whenever there is an a, it balanced by an other a (after some b's).
- Gives the factor of the form (ab*ab*)
- The words can have as many factors of this from as it wants. It can end an a or ab.

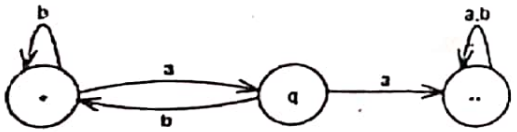
Scanned with CamScanner

Theorem 12

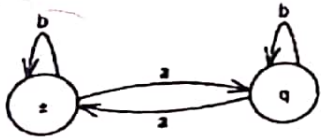
Example

- These two languages can also be defined by FA (Kleen's theorem)

FA$_1$



FA$_2$



Theorem 12

- The first step in building the machine (and regular expression) for $L_1 \cap L_2$ is to find the machines that accept the complementary languages L_1' and L_2'.
- The English description of these languages is:

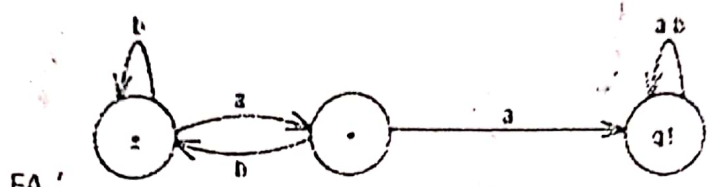
  L_1' = all strings that do not contain the substring a

  L_2' = all strings having an odd number of a's

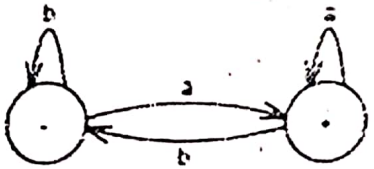
Scanned with CamScanner

Theorem 12

- The machines for these languages are



FA_1'

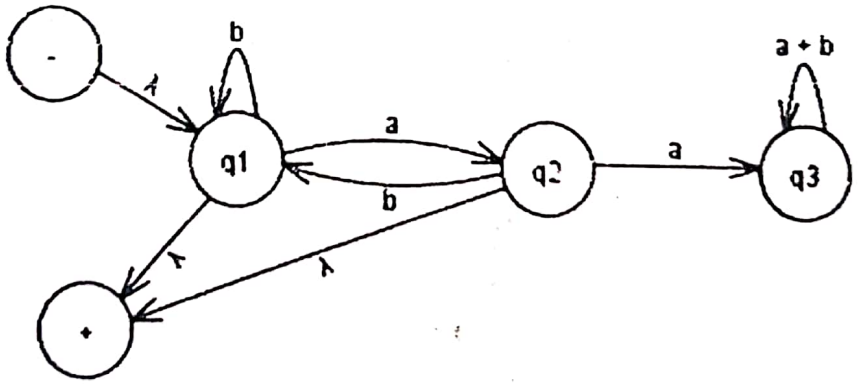


FA_2'

Scanned with CamScanner

Theorem 12

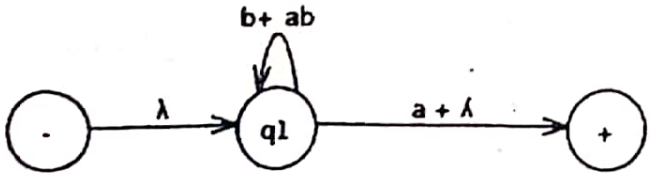
- Recall that how we go through stages of transition graphs with edges labeled by regular expressions. Thus, FA_1' becomes:



Scanned with CamScanner

Theorem 12

- State q_3 is part of no path from - to +, so it can be dropped.
- We ned to join incoming a edge with both outgoing edges(b to q1 and q2 to +).
- When we add the two loops, we get b + ab and the sum of the two edges from q1 to + is a + Λ so the machine looks like



Theorem 12

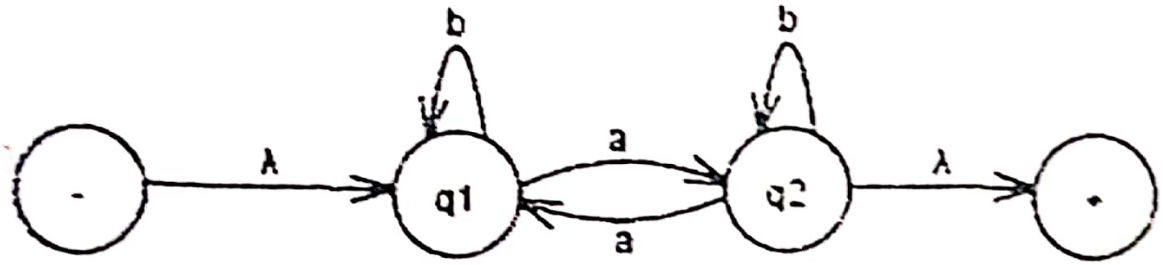
- The last step is to bypass state q_1.
- Concentrate the incoming Λ label with the loop label starred (b + ab)*
- Concentrated with outgoing (a + Λ) to produce one edge from – to + with the regular expression for L_1'

$$r_1' = (b + ab)^*(a + \Lambda)$$

Scanned with CamScanner

Theorem 12

- Lets do the same thing for the language L_2
- FA2 becomes

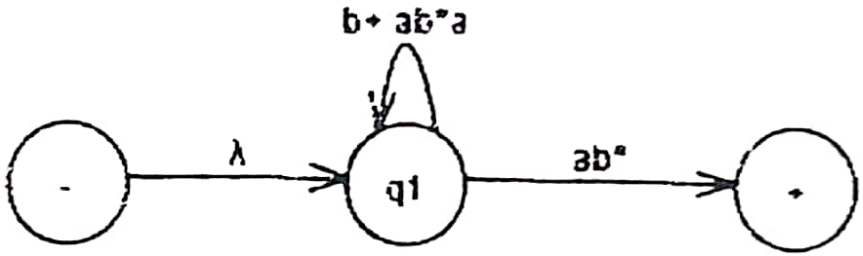


- Simplify:
- Eliminate state q_2
- There is one incoming edge a loop, and two outgoing edges,
- Now to replace them with only two edges:
- The path $q_1 - q_2 - q_2 - q_1$ becomes a loop at q_1
- The path $q_1 - q_2 - q_2 - +$ becomes an edge from q1 to +.

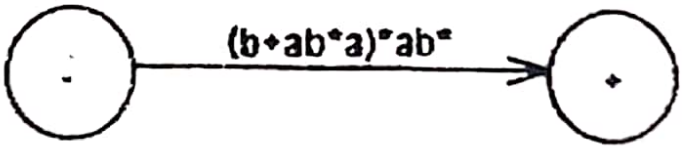
Scanned with CamScanner

Theorem 12

- When state q_2 is bypassed and adding the two loop labels we get,



- We can eliminate state q_1 we get,



Scanned with CamScanner

Theorem 12

- It gives the regular expression

  $r_2' = (b + ab^*a)^*ab^*$

- This is one of the regular expression that define the language of all words with an odd number of a's.
- Another expression that define the same language
- Add b*a in the front of regular expression for L1

We get

b*ab*(ab*ab*)*

Theorem 12

- This works because words with an odd number of a's can be interpreted as b*a in front of words with an even number of a's.
- The fact that these two different regular expressions define the same language is not obvious.

Scanned with CamScanner

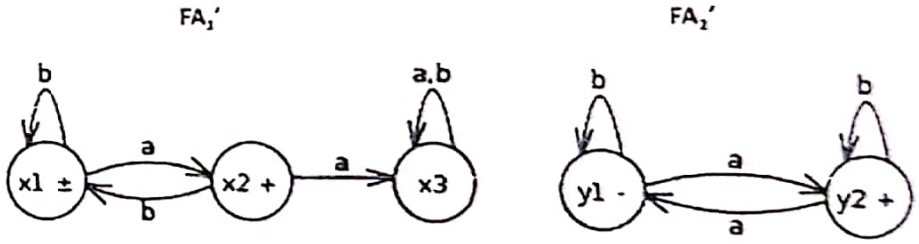
- As we have regular expression for L_1' and L_2'
- We can write the regular expression for $L_1'+L_2'$

$r_1' + r_2' = (b + ab)^*(a + \Lambda) + (b + ab^*a)^*ab^*$

- Make this regular expression in to FA
- so that we can take its complement to get the FA that defines $L_1 \cap L_2$.

Theorem 12

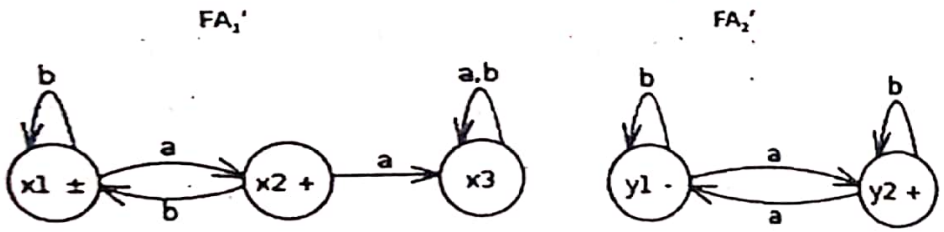
- Let us label the states in the two machines for FA_1' and FA_2'.



- The start states are x_1, and y_1 and the final states are x_1, x_2, and y_2.

Scanned with CamScanner

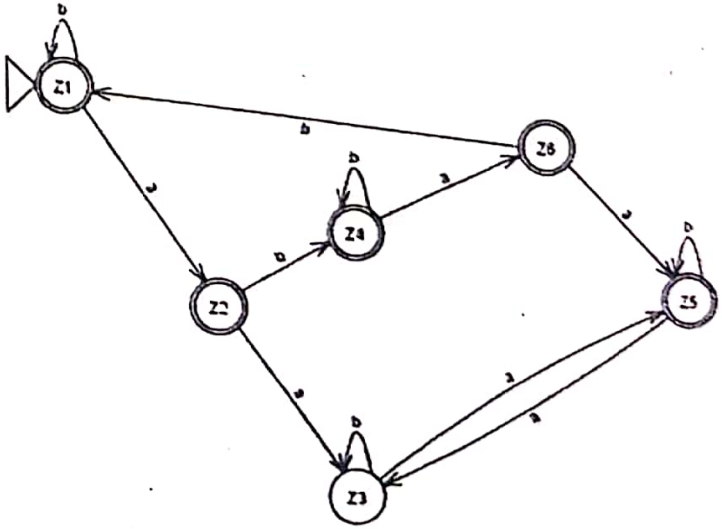
Theorem 12...



Old States	Reading at a	Reading at b
z1-+≡(x1,y1)	(x2,y2)≡z2	(x1,y1)≡z1
Z2+≡(x2,y2)	(x3,y1)≡z3	(x1,y2)≡z4
z3≡(x3,y1)	(x3,y2)≡z5	(x3,y1)≡z3
Z4+≡(x1,y2)	(x2,y1)≡z6	(x1,y2)≡z4
Z5+≡(x3,y2)	(x3,y1)≡z3	(x3,y2)≡z5
Z6+≡(x2,y1)	(x3,y2)≡z5	(x1,y1)≡z1

Theorem 12

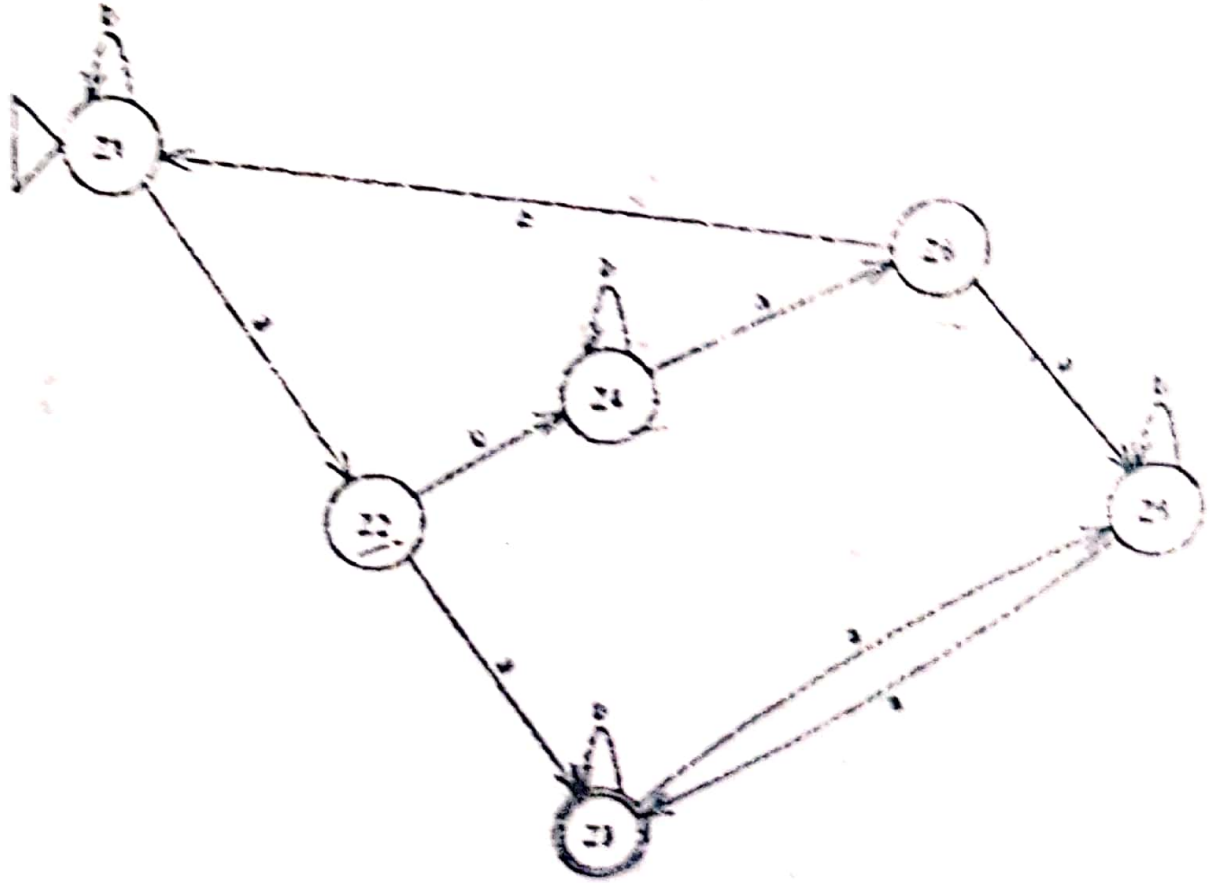
- The union machine



Scanned with CamScanner

Theorem 12

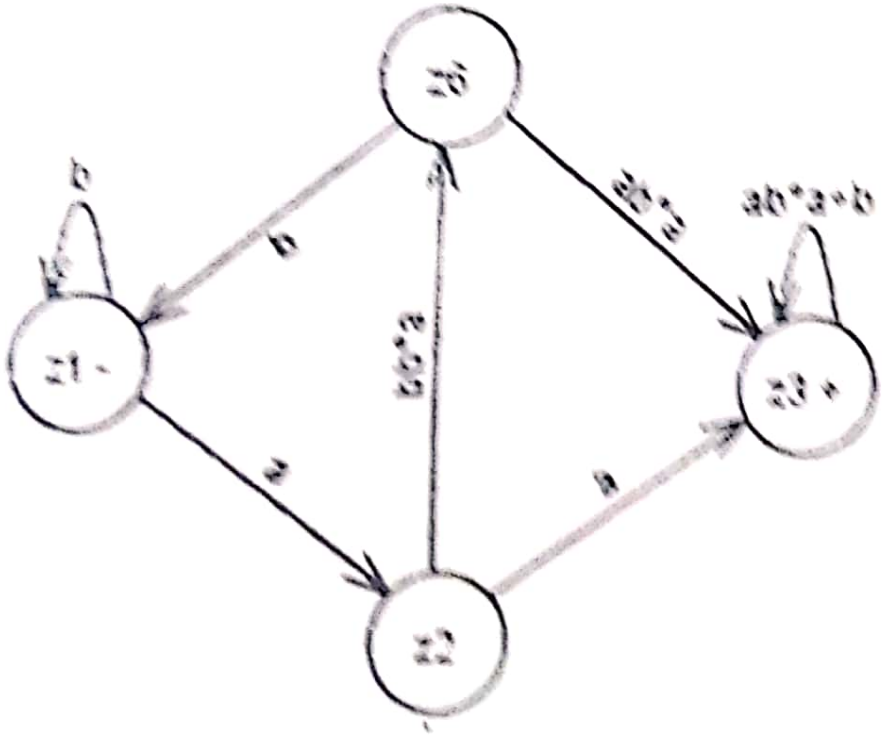
- This FA can accept the language $L_1' + L_2'$
- Reverse the status of each state from final to non-final and vice versa, an FA is going to be produced for the language $L_1 \cap L_2$ after taking the complement again.



Scanned with CamScanner

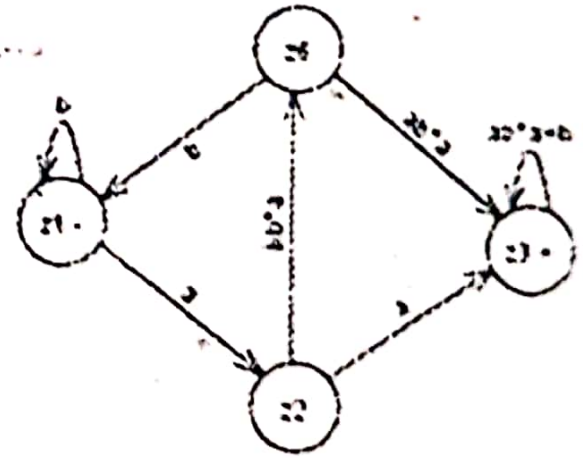
Theorem 12

- Bypassing z_4 and z_5 gives

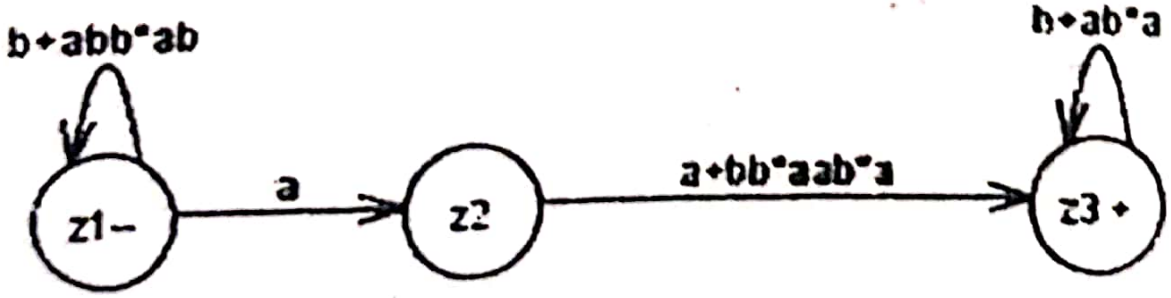


Scanned with CamScanner

Theorem 12



- Then bypassing z_3 gives



- So the whole machine reduces to regular expression

$$(b + abb^*ab)^*a(a + bb^*aab^*a)(b + ab^*a)^*$$

Scanned with CamScanner

Theorem 12

$$(b + abb^*ab)^*a(a + bb^*aab^*a)(b + ab^*a)^*$$

- As it stands, there are four factors (the second is just an a and the first and fourth are starred).
- Every time we use one of the options from the two end factors we incorporate an even number of a's into the word (either none or two).
- The second factor gives us an odd number of a's (exactly one).

Scanned with CamScanner

Theorem 12

- Are all the words in $L_1 \cap L_2$ included in the language of this expression?
  - YES
  - Look at any word that is in $L_1 \cap L_2$
  - It has an even number a's and a double a somewhere

Non-regular languages

- Many Languages can be defined using FA's.
- These languages have had many different structures, they took only a few basic forms: languages with required substrings, languages that forbid some substrings, languages that begin or end with certain strings,
- languages with certain even/odd properties, and so on.

Non-regular languages

- Lets take a look to some new forms, such as the language PALINDROME or the language PRIME of all words a^p where p is a prime number.
- Neither of these is a regular language.
- They can be described in English, but they cannot be defined by an FA.
- Conclusion: More powerful machines are needed to define them

Non-regular languages

Definition

A language that cannot be defined by a regular expression is called a non-regular language.

Scanned with CamScanner

Non-regular languages

- Cannot be accepted by any FA or TG. (Kleene's theorem,)
- All languages are either regular or nonregular, but not both of them.

Non-regular languages

- Consider a simple case. Let us define the language L.
- L = {a, ab, aabb, aaabbb, aaaabbbb, aaaaabbbbb . ..}
- We could also define this language by the formula

$$L = \{a^n b^n \text{ for } n = 0\ 1\ 2\ 3\ 4\ 5\ \ldots\}$$

Or short for

$$L = \{a^n b^n\}$$

Non-regular languages

- Please not that when the range of the abstract exponent n is unspecified we mean to imply that it is 0,1,2,3, ...
- Lets show that this language is nonregular.
- It is a subset of many regular languages, such as a*b*, which, however, also includes such strings as aab and bb that $\{a^n b^n\}$ does not.

Non-regular languages

- Suppose this language is regular and some FA must be developed that accepts it.
- Let us picture one of these FA's .This FA might have many states.
- Say that it has 95 states, just for the sake of argument.
- Yet we know it accepts the word $a^{96}b^{96}$. The first 96 letters of this input string are all a's and they trace a path through this hypothetical machine.

Scanned with CamScanner

Non-regular languages

- Because there are only 95 states ,The path cannot visit a new state with each input letter read.
- The path returns to a state that it has already visited at some point.
- The first time it was in that state it left by the a-road.
- The second time it is in that state it leaves by the a-road again.

Non-regular languages

- Even if it only returns once we say that the path contains a circuit in it.
  (A circuit is a loop that can be made of several edges.)
- The path goes up to the circuit and then it starts to loop around the circuit, maybe 0 or more times.
- It cannot leave the circuit until, b is read in.

Non-regular languages

- Then the path can take a different turn. In this hypothetical example the path could make 30 loops around a three-state circuit before the first b is read.
- After the first b is read, the path goes off and does some other stuff following b edges and eventually winds up at a final state where the word $a^{96}b^{96}$ is accepted.
- Let us, say that the circuit that the a-edge path loops 'around has seven states in it. The path enters the circuit, loops around it continuously and then goes off on the b-line to a final state.

Non-regular languages

- What will happen to the input string $a^{96+7}b^{96}$? Just as in the case of the input string $a^{96}b^{96}$,
- This string would produce a path through the machin that would walk up to the same circuit (reading in o a's) and begin to loop around it in exactly the same way.
- However, the path for $a^{96+7}b^{96}$ loops around this circu one more time than the path for $a^{96}b^{96}$ --precisely on extra time.

Scanned with CamScanner

Nonregular language

- Both paths, at exactly the same state in the circuit, begin to branch off on the b-road.
- Once on the b-road, they both go the same 96 b-steps and arrive at the same final state.
- But this would mean that the input string $a^{103}\,b^{96}$ is accepted by this machine.
- But that string is not in the language L = $\{a^n b^n\}$.

Non-regular languages

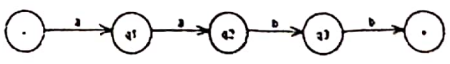
- This is a contradiction. We assumed that we were talking about an FA that accepts exactly the words in L and then we were able to prove that the same machine accepts some word that is not in L.
- This contradiction means that the machine that accepts exactly the words in L does not exist.
- In other words, L is nonregular.

Non-regular languages

- Let us review what happened.
- We chose a word in L that was so large (had so many letters) that its path through the FA had to contain a circuit.
- Once we found that some path with a circuit could reach a final state, we asked ourselves what happens to a path that is just like the first one,
- But that loops around the circuit one extra time and then proceeds identically through the machine.

Non-regular languages

- Suppose the following FA for a^2b^2



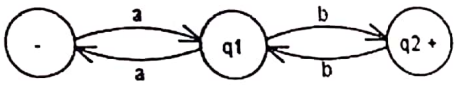
- Only those used in the path of a^2b^2.

Scanned with CamScanner

Non-regular languages

Consider the following FA with Circuit....

However, the path for $a^{13}\,b^{9}$ still has four more a-steps to take, which is one more time around the circuit, and then it follows the nine b-steps.



Non-regular languages

- Let us return to.
- With our first assumptions we made above (that there were 95 states and that the circuit was 7 states long), we could also say that $a^{110}b^{96}$, $a^{117}b^{96}$, $a^{124}b^{96}$ ... are also accepted by this machine.
- All can be written the form

$$a^{96}(a^{7})^{m}b^{96}$$

- where m is any integer 0,1,2,3 ..... If m is 0, the path through this machine is the path for the word $a^{96}b^{96}$.

Non-regular languages

- If m is (1, that looks the same, but it loops the circuit one more time.
- If m 2, the path loops the circuit two more times.
- In general, $a^{96}(a^{7})^{m}b^{96}$ loops the circuit exactly m more times.
- After doing this looping it gets off the circuit at exactly the same place $a^{96}b^{96}$ does and proceeds along exactly the same route to the final state.
- All these words, though not in L, must be accepted.

Non-regular languages

- Suppose that we had considered a different machine to accept the language L, a machine that has 732 states.
- When we input the word $a^{733}\,b^{733}$,
- the path that the a's take must contain a circuit.
- The word $a^{9999}b^{9999}$ also must loop around a circuit i its a-part of the path.

Scanned with CamScanner

Nonregular language

- Suppose the circuit that the a-part follows has 101 states.
- Then $a^{733+101}\ b^{733}$ would also have to be accepted by this machine, because
- its path is the same in every detail except that it loops the circuit one more time.
- This second machine must also accept some strings, that are not in L:

$a^{834}\ b^{733}$ $a^{935}\ b^{733}$ $a^{1036}\ b^{733}$ .....

$= a^{733}\ (a^{101})^{m}\ b^{733}$

Scanned with CamScanner